

Chapitre 1 : scalable systems

1/Rappel

La normalisation vise à organiser les données pour éviter les problèmes. Ses objectifs principaux sont de réduire la redondance des données, d'améliorer leur intégrité en évitant les incohérences, et de faciliter la maintenance (mises à jour, insertions, suppressions).

Les formes normales les plus courantes sont :

- **1NF** : chaque colonne contient une seule valeur, sans groupes répétitifs.
- **2NF** : la table est en 1NF et toutes les colonnes non-clés dépendent de toute la clé primaire.
- **3NF** : la table est en 2NF et aucune colonne non-clé ne dépend d'une autre colonne non-clé.

Exemple de la 1NF

D Client	Nom	Adresses
1	Dupont	10 Rue A, Paris
2	Martin	25 Rue B, Lyon; 30 Rue C, Marseille

Avant la **1NF**, la table contient une colonne *Adresses* non atomique, car un client peut avoir plusieurs adresses dans une seule cellule (séparées par un point-virgule). Cela provoque des problèmes, notamment l'impossibilité de rechercher facilement un client par numéro de rue.

Après la **1NF**, les données sont réorganisées pour que chaque champ contienne une seule valeur. Les informations sont séparées en deux tables :

- une table **Client** (ID, ClientNom),
- une table **Adresse** reliant chaque client à une adresse unique.

ID Adresse	ID Client	Adresse
101	1	10 Rue A, Paris
102	2	25 Rue B, Lyon
103	2	30 Rue C, Marseille

ID Client	Nom
1	Dupont
2	Martin

Ainsi, chaque adresse est stockée sur une ligne distincte, ce qui respecte la 1NF et facilite les recherches et la gestion des données.

Deuxième forme normale (2NF)

La **2NF** s'applique aux tables déjà en 1NF. Pour qu'une table soit en 2NF, chaque colonne non-clé doit dépendre **entièrement** de la clé primaire complète, et non d'une partie seulement.

Exemple :

- Table originale avec clé primaire composite (**ID_étudiant**, **ID_cours**) :
 - Attributs comme **nom_cours** et **nom_instructeur** dépendent seulement de **ID_cours** (partie de la clé), ce qui viole la 2NF.
- Solution : créer une table **Cours** (**ID_cours**, **nom_cours**, **nom_instructeur**) et conserver (**ID_étudiant**, **ID_cours**) dans la table originale.

Cela élimine la dépendance partielle et respecte la 2NF.

Troisième forme normale (3NF)

La **3NF** élimine les dépendances transitives : un attribut non-clé ne doit pas dépendre d'un autre attribut non-clé.

Exemple :

- Table originale : (**PatientID**, **DoctorID**, **DoctorName**)
 - **DoctorName** dépend de **DoctorID**, qui dépend de **PatientID** → dépendance transitive.
- Solution : créer une table **Docteurs** (**DoctorID**, **DoctorName**) et conserver (**PatientID**, **DoctorID**) dans la table originale.

Cela garantit que chaque attribut non-clé dépend uniquement de la clé primaire.

2/Systèmes évolutifs (Scalable Systems)

C'est quoi un système scalable (évolutif)?

Un système évolutif peut gérer une augmentation de charge ou de demandes utilisateurs en ajoutant des ressources (serveurs, puissance de calcul) sans perte significative de performance. Il permet au système de croître et de s'adapter aux besoins changeants tout en maintenant efficacité, fiabilité et expérience utilisateur constante.

3/Caractéristiques principales :

- **Gère une charge accrue** (plus d'utilisateurs, de données ou de transactions).
- Peut **ajouter des ressources** (serveurs, puissance de calcul) sans remplacer le système.
- **Maintient la performance** (temps de réponse stable) malgré l'augmentation de la charge.
- **S'adapte aux évolutions (changements)** pour continuer à répondre aux besoins des utilisateurs et aux objectifs de l'entreprise.

4/Importance de la scalabilité :

- Permet la croissance de l'entreprise sans limitation technologique.
- Réduit les coûts en ajoutant les ressources progressivement.
- Assure fiabilité et performance constante même sous forte demande.

- Permet de gérer des pics imprévus de trafic ou de demande.

5/ 7 Stratégies pour scaler une base de données :

Lien vidéo : [7 Must-know Strategies to Scale Your Database](#)

1. Indexation (B-Tree Index)

- Accélère les opérations de lecture, insertion et suppression.
- Attention : améliore les lectures mais peut ralentir les écritures car l'index doit être mis à jour.
- Clé : choisir les bons champs à indexer pour un équilibre optimal performance lecture/écriture.

2. Materialized Views (Vues matérialisées)

- Ce sont des **instantanés pré-calculés** des données pour un accès plus rapide.
- Doivent être **rafraîchies régulièrement** pour rester à jour.
- Exemple d'usage : plateformes BI comme Tableau.

Exemple SQL :

```
CREATE MATERIALIZED VIEW user_purchase_summary AS
SELECT
  u.id AS user_id,
  COUNT(p.id) AS total_purchases,
  SUM(CASE WHEN p.status = 'cancelled' THEN 1 ELSE 0 END) AS
  cancelled_purchases
FROM users u
JOIN purchases p ON p.user_id = u.id
GROUP BY u.id;
```

- Cette vue matérialisée résume pour chaque utilisateur le nombre total d'achats et d'achats annulés.

3. Dénormalisation

- La **dénormalisation** consiste à stocker des données redondantes pour simplifier les requêtes et accélérer l'accès aux données.
- **Exemple** : sur les réseaux sociaux comme **Facebook**, les posts et informations des utilisateurs sont stockés dans la même table.
- **Avantage** : réduit les **joins complexes**, ce qui accélère l'affichage des flux utilisateurs.

4. Scalabilité verticale (Vertical Scaling)

- **Principe** : Ajouter des ressources (CPU, RAM, stockage) au serveur de base de données existant pour gérer une charge plus élevée.
- **Exemple** : un marketplace en ligne améliore son serveur avec plus de CPU, de RAM et de stockage pour traiter plus de transactions et répondre plus vite aux requêtes.

- **Avantages** : facile à mettre en place, améliore immédiatement les performances sans modifier l'architecture de l'application.
- **Limite** : ne gère pas la redondance ; une panne du serveur peut arrêter la base de données.

5. Caching

- **Principe** : Stocker les données fréquemment consultées dans une mémoire plus rapide pour réduire la charge sur la base et accélérer les réponses.
- **Exemple** : Netflix utilise un cache pour récupérer les métadonnées des films plutôt que d'interroger la base à chaque fois.
- **Implémentation** : en mémoire avec Redis, Memcache, ou au niveau de l'application.
- **Limite** : il faut gérer la **validation du cache** pour éviter les données obsolètes, via des mises à jour périodiques ou déclenchées par les événements.

6. Réplication (Replication)

- **Principe** : Créer des copies de la base principale sur plusieurs serveurs pour améliorer la disponibilité, répartir la charge et renforcer la tolérance aux pannes.
- **Types** :
 - **Synchronisée** : la copie est mise à jour en même temps que la base principale → cohérence immédiate, mais latence possible.
 - **Asynchrone** : la base principale n'attend pas la confirmation des copies → meilleure performance, mais cohérence temporairement décalée.
- **Avantages** : améliore les lectures et la disponibilité.
- **Limites** : complexité pour maintenir la cohérence et augmentation du stockage et de la maintenance.

7. Sharding

- **Principe** : Diviser une grande base de données en morceaux plus petits appelés **shards** pour mieux gérer la charge.
- **Exemple** : Instagram répartit les données des utilisateurs par **user ID**, chaque utilisateur ayant ses données sur un shard spécifique.
- **Avantages** : efficace pour la scalabilité horizontale et la gestion de grandes bases de données.
- **Limites** : complexité dans la conception et la gestion, requêtes inter-shards compliquées, et re-sharding coûteux lorsqu'un shard devient déséquilibré.

Conclusion

Pour scaler efficacement une base de données, il faut équilibrer **amélioration des performances** et **gestion de la complexité**. L'application judicieuse des différentes stratégies permet à la base de répondre aux besoins croissants de l'application.

Chapitre 2 : Design d'un système scalable (de 0 à des millions d'utilisateurs)

Lien vidéo : [YouTube: System Design: Scale System From Zero To Million Users | #systemdesign](#)

1/Scalabilité verticale :

- Consiste à ajouter des ressources matérielles (CPU, RAM) à un seul serveur.
- Simple à mettre en œuvre, mais limitée par la capacité maximale du serveur.
- Elle possède donc une **limite physique stricte** et ne suffit pas pour une très grande montée en charge.

2/Scalabilité horizontale :

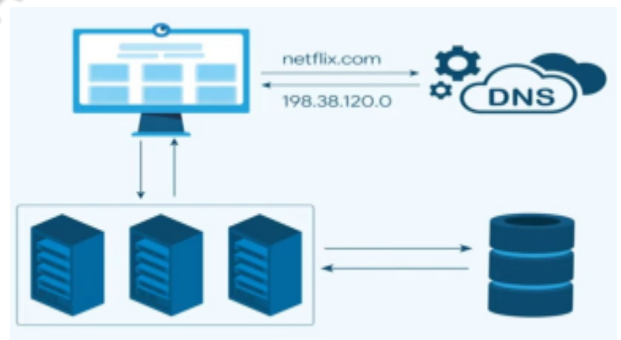
- Consiste à ajouter plusieurs serveurs dans un même pool de ressources.
- C'est la méthode privilégiée pour concevoir des applications à grande échelle.
- La répartition des requêtes des utilisateurs entre les serveurs est assurée par un **load balancer**, qui décide quel serveur traitera chaque requête.

Explication de l'image

Le **DNS** est un système qui traduit les noms de domaine lisibles par l'homme (ex. *netflix.com*) en adresses IP numériques (ex. *198.38.120.0*).

Fonctionnement :

- L'utilisateur saisit *netflix.com* dans le navigateur.
- Le DNS est interrogé pour trouver l'adresse IP correspondante.
- Le DNS renvoie l'adresse IP du serveur.
- L'ordinateur se connecte ensuite au serveur à cette adresse.

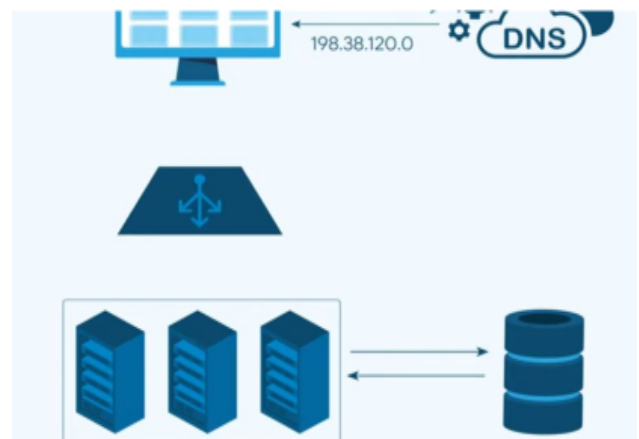


Ce mécanisme permet d'accéder aux sites web via des noms simples au lieu de retenir des adresses IP complexes.

Explication de l'image -Résolution DNS inverse (Reverse DNS)

Cette représentation illustre la **résolution DNS inverse**, où l'on cherche le nom de domaine associé à une adresse IP.

- Au lieu de *nom de domaine* → *adresse IP*, on a : **adresse IP** → **nom de domaine**.
- Elle utilise des enregistrements DNS spécifiques appelés **PTR (Pointer)**.



Utilisations principales :

- Vérification des serveurs de messagerie.
- Analyse des logs et diagnostics réseau.
- Sécurité et validation de l'authenticité des serveurs.

En résumé, contrairement à la résolution DNS classique utilisée par les utilisateurs, la résolution inverse est surtout importante pour l'administration réseau et la sécurité

3/Scalabilité horizontale — Rôle du Load Balancer

Dans une architecture de scalabilité horizontale, l'utilisateur communique uniquement avec le **load balancer**. Celui-ci reçoit la requête et la redirige vers le serveur le moins chargé. Le serveur traite la requête et renvoie la réponse au load balancer, qui la transmet ensuite à l'utilisateur.

Avantages :

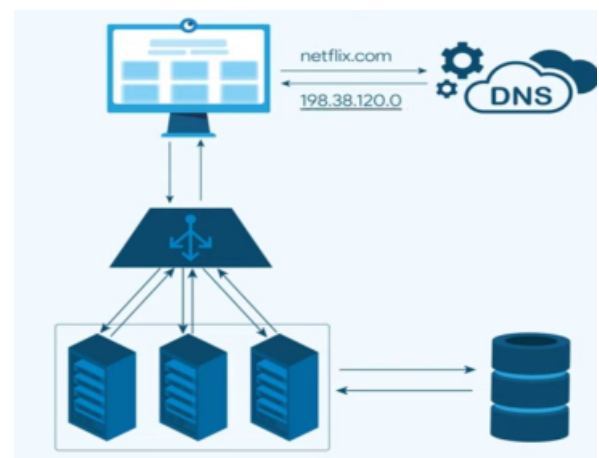
- Répartition efficace de la charge entre les serveurs.
- Les utilisateurs ne connaissent que l'IP du load balancer, pas celles des serveurs internes.
- Amélioration de la sécurité en réduisant les risques d'attaques directes sur les serveurs.

Explication de l'image -Fonctionnement du DNS

Le **DNS** est le système qui traduit les noms de domaine lisibles par l'homme (ex. *netflix.com*) en adresses IP numériques (ex. *198.38.120.0*).

Fonctionnement :

- L'utilisateur saisit *netflix.com* dans le navigateur.
- L'ordinateur interroge un serveur DNS pour obtenir l'adresse IP correspondante.
- Le DNS renvoie l'adresse IP du serveur.
- L'ordinateur se connecte alors à cette adresse pour charger le site.



Le DNS agit comme un **annuaire Internet**, permettant d'utiliser des noms simples au lieu de mémoriser des adresses IP complexes.

4/Problème d'une base de données unique et réplication

Avec une seule instance de base de données, une panne (ex. coupure de courant) rend le système inaccessible.

La solution consiste à appliquer une **scalabilité horizontale des bases de données**, appelée **réplication**.

- Une **instance master** gère uniquement les écritures.

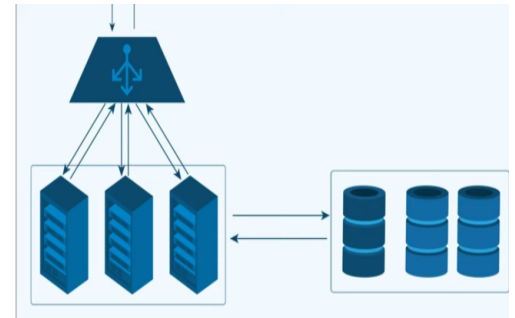
- Les **instances slave** traitent uniquement les lectures.
- Les données sont synchronisées entre le master et les slaves.

Cette approche améliore la disponibilité et la tolérance aux pannes.

Explication de l'image - Architecture client-serveur avec load balancer

L'image illustre une architecture scalable avec **répartition de charge** :

- Les requêtes arrivent sur un **load balancer**, qui les distribue intelligemment.
- Elles sont traitées par plusieurs **serveurs applicatifs**, ce qui améliore les performances, la disponibilité et la tolérance aux pannes.
- Les serveurs communiquent avec des **bases de données** (répliquées ou en cluster) pour lire et écrire les données.



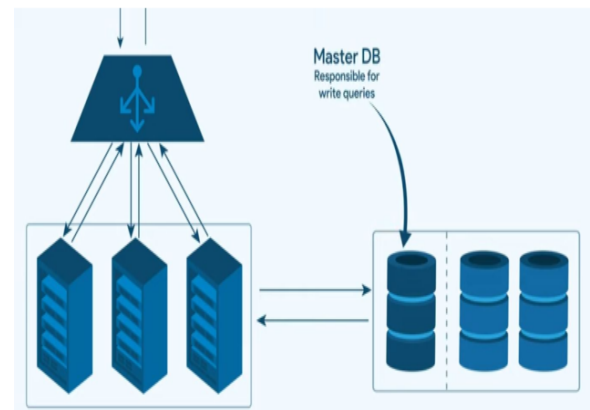
C'est une architecture courante pour les applications web à forte charge.

Explication de l'image -Rôle du Master DB dans une base répliquée

Dans une architecture de base de données

Master-Slave (Primary-Replica) :

- Le **Master DB** est la base principale et la **source unique de vérité**.
- Il gère **toutes les requêtes d'écriture** : INSERT, UPDATE, DELETE et modifications de structure.
- Les changements sont ensuite **répliqués vers les bases Slave**, qui servent principalement les lectures.



Intérêt de cette architecture :

- Améliore les **performances** en séparant lectures et écritures.
- Assure une meilleure **disponibilité** et une **sauvegarde** via les répliques.
- Permet l'**évolutivité** en ajoutant des Slaves pour supporter plus de lectures.

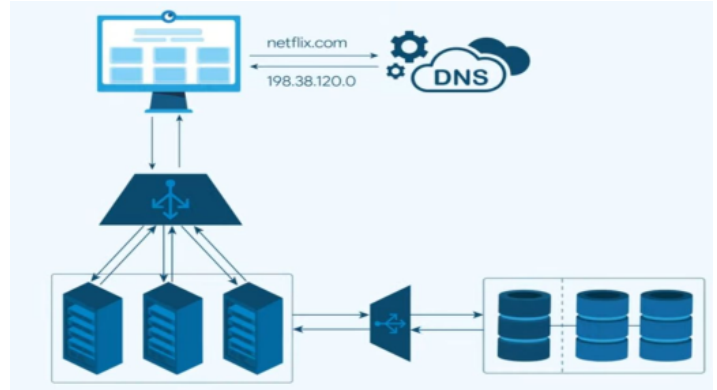
Le Master est donc un composant central et critique, garantissant la cohérence et l'intégrité des données.

5/Scalabilité horizontale — Réplication et load balancer pour bases de données

- La **réplication de bases de données** améliore les performances, la fiabilité et la disponibilité.
- Problème : les serveurs doivent choisir à quel **slave** envoyer les requêtes.
- Solution : utiliser un **load balancer pour les bases de données**, qui répartit automatiquement les requêtes entre les réplicas.

Explication de l'image - Architecture complète d'un site web à grande échelle

- **Utilisateur / Navigateur** : saisit un nom de domaine (ex. *netflix.com*).
- **DNS** : traduit le nom de domaine en adresse IP (ex. 198.38.120.0).
- **Load Balancer** : reçoit la requête et la distribue entre plusieurs serveurs applicatifs pour équilibrer la charge.
- **Serveurs applicatifs** : traitent les requêtes et communiquent avec la base de données pour lire ou écrire des informations.
- **Base de données** : stocke toutes les données nécessaires (films, utilisateurs, historiques).
- **Switch / Réseau interne** : assure la communication rapide et sécurisée entre serveurs et base.



Résumé global :

Utilisateur → DNS → Load Balancer → Serveurs → Base de données

Cette architecture garantit **rapidité, disponibilité et scalabilité** pour des sites à très fort trafic.

6/Problèmes et solutions pour la base de données et la performance

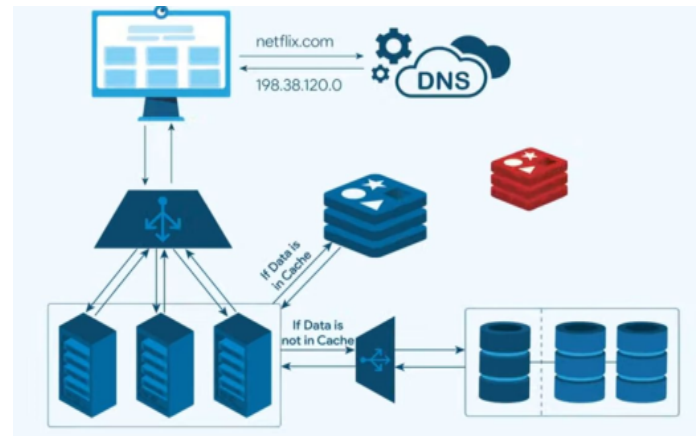
- **Problème 2** : Si le **Master DB** tombe en panne, le système ne peut plus traiter les écritures.
 - **Solution** : un **Slave DB** est automatiquement promu en Master pour reprendre les écritures.
- **Problème 3** : Les requêtes directes vers la base de données sont coûteuses et augmentent le temps de réponse.
 - **Solution** : installer un **cache** entre le serveur et la base de données.
 - Le cache stocke temporairement les données fréquemment utilisées sous forme de **paires clé-valeur** (ex. { 'name', 'devsense' }), permettant au serveur de vérifier d'abord le cache avant d'interroger la base.

Rôle du cache

- Si les données sont présentes dans le **cache**, elles sont récupérées directement de là.
- Si elles ne s'y trouvent pas, le serveur interroge la **base de données**, puis stocke les données récupérées dans le cache pour les futures requêtes.

Explication de l'image -Fonctionnement du DNS avec cache

- **DNS de base** : traduit un nom de domaine (ex. *netflix.com*) en adresse IP (ex. 198.38.120.0).
- **Avec cache** : l'ordinateur vérifie d'abord son **cache DNS local** ; si l'adresse y est, elle est utilisée immédiatement.
- **Sans cache** :
 1. L'ordinateur ne trouve rien dans le cache.
 2. Il interroge le serveur DNS du FAI, puis éventuellement d'autres serveurs DNS.
 3. Le DNS renvoie l'adresse IP correspondante.
 4. L'ordinateur stocke cette information dans son cache pour les prochaines requêtes.



Le cache permet ainsi d'accélérer les résolutions DNS et de réduire la charge sur les serveurs DNS.

7/Types et aspects importants du cache

Pour gérer un cache efficacement, il faut connaître :

- **Eviction Policy** : règles pour décider quelles données supprimer lorsque le cache est plein.
- **Expiration Policy** : durée de vie des données dans le cache avant qu'elles ne soient considérées comme obsolètes.
- **Consistency Requirements** : exigences de cohérence entre le cache et la base de données pour garantir que les données sont correctes et à jour.

Chapitre 3 :Micro Services Architecture

Lien :<https://ncube.com/microservices-vs-monolithic-which-architecture-suits-best-for-your-project>

1/Rappel : Deux styles d'architecture

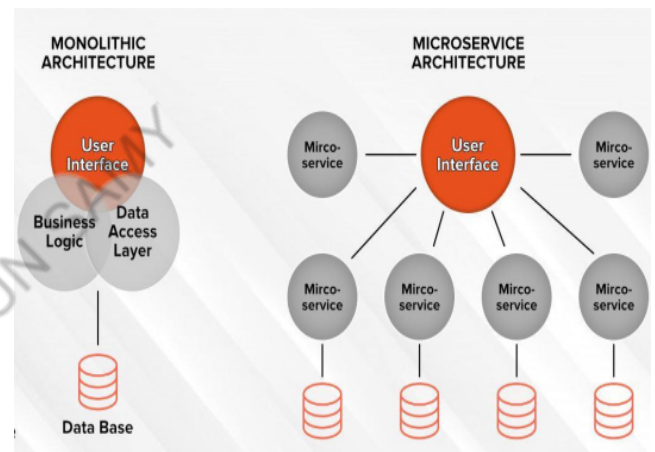
1. Monolithique

- Développée en un **seul bloc** avec généralement une seule technologie.
- Déployée sur **un seul serveur d'application**.
- Composants typiques :
 - Conteneur web
 - Framework assurant l'**inversion de contrôle**
- L'application est organisée en **plusieurs couches** (ex. présentation, logique métier, données).

Explication de l'image -Comparaison Monolithique vs Microservices

1. Architecture Monolithique

- Application regroupée en un **seul bloc** avec :
 - **UI** : interface utilisateur
 - **Business Logic** : logique métier
 - **Data Access Layer** : accès aux données
- **Une seule base de données**
- **Avantages** : simple à développer, tester et déployer au début.
- **Inconvénients** : difficile à maintenir, faire évoluer et scaler ; toute modification nécessite souvent un redéploiement complet.



2. Architecture Microservices

- Application **découpée en petits services indépendants**.
- Chaque microservice :
 - Gère une **fonctionnalité spécifique**
 - Peut avoir **sa propre base de données**
 - Communique via **API**
- **Avantages** : développement, déploiement et scalabilité indépendants ; équipes multiples possibles.
- **Inconvénients** : plus complexe à gérer (communication inter-services, données distribuées, orchestration) ; nécessite une maturité DevOps.

Résumé global :

Microservices = modularité, flexibilité et scalabilité accrues par rapport au monolithe, adaptés aux systèmes modernes à forte charge.

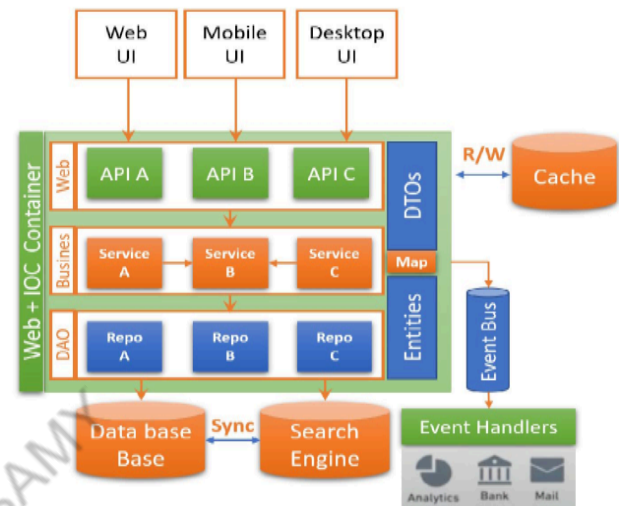
2/Architecture Monolithique — Organisation en couches

- **Couche Web** : web services, GraphQL, interface avec l'utilisateur.

- **Couche Business** : logique métier et traitement principal de l'application.
- **Couche DAO (Data Access / Repository)** : utilise des frameworks comme **Hibernate** ou **Spring Data** pour faciliter l'utilisation de **JPA** et le mapping objet-relationnel (O/R).
- **Couche Base de données** : stockage et gestion des données.

Explication de l'image -Architecture Microservices Moderne

- **UI (Interface Utilisateur)** : Web, Mobile, Desktop ; communique avec les API via des appels réseau.
- **API** : exposent des services spécifiques pour chaque interface ; gèrent une partie de la logique métier.
- **Services** : implémentent la logique métier de manière indépendante; peuvent être mis à l'échelle séparément.
- **Repositories (DAO)** : accès aux données, traduisent les requêtes des services en opérations sur la base.
- **Modélisation** : DTOs pour le transfert de données entre couches, Map Entities pour convertir objets métier ↔ entités de base.
- **Infrastructure de données** :
 - Base de données principale
 - Moteur de recherche (ex. Elasticsearch) pour requêtes rapides
 - Cache pour accélérer l'accès aux données fréquentes
- **Bus d'événements** : communication asynchrone entre services via Event Bus et Event Handlers (analytics, transactions, notifications).



Résumé global :

Architecture distribuée et modulaire où les interfaces consomment des API, les services métier sont découplés et les données sont accessibles via base, cache et moteur de recherche. Cette structure favorise **scalabilité**, **résilience** et **maintenabilité**.

3/Microservices et bonnes pratiques

- **Entités (tables)** :
 - Mauvaise pratique : les utiliser directement dans toutes les couches.
 - Bonne pratique :
 - Couche **repository** et **métier** : entités utilisées pour la logique et la persistance.
 - Couche **web** : utiliser des **DTOs** (Data Transfer Objects) via des **mappers** pour ne transférer que les données nécessaires, sans annotations de base de données.
- **DAO (Data Access Object)** : pattern pour faire le lien entre la couche métier et la couche de persistance.
- **Exemples d'applications** : gestion de paie, gestion des stocks, comptabilité.

- **Exemple d'architecture orientée services / microservices :**

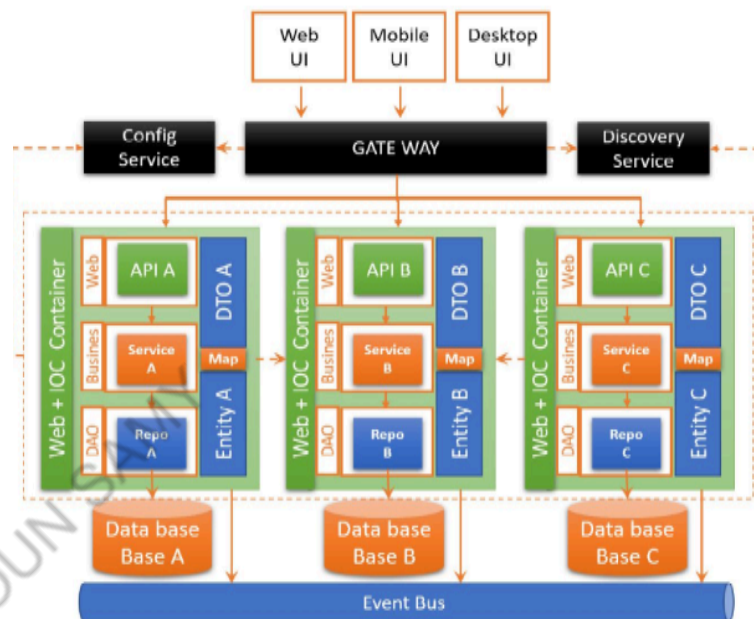
- Module e-commerce
- Module inventaire
- Module gestion client
- Module facturation
- Module commande

Explication de l'image -Architecture Microservices typique

1. **Clients (UI) :** Web, Mobile, Desktop ; communiquent via un point d'entrée unique.

2. **Composants centraux :**

- **API Gateway :** routage des requêtes, authentification, load balancing.
- **Config Service :** configuration centralisée pour tous les services.
- **Discovery Service :** localisation dynamique des microservices dans un environnement conteneurisé.



3. **Microservices :** indépendants, conteneurisés, avec structure interne typique :

- **API → Service → DTO/Mapper → Repository/DAO → Entity → Base de données**
- Utilisation de **DTOs** pour isoler les entités internes.

4. **Bases de données :** chaque microservice possède sa **DB dédiée** (pattern Database per Service) pour éviter le couplage fort.

5. **Event Bus :** communication asynchrone entre microservices via événements (ex. RabbitMQ, Kafka) pour maintenir la cohérence éventuelle.

Avantages : scalabilité indépendante, déploiement séparé, résilience, flexibilité technologique.

Patterns clés : API Gateway, Service Discovery, Database per Service, Event-Driven Architecture.

4/API Gateway et configuration centralisée

- **API Gateway :** point d'entrée unique vers l'application, acheminement des requêtes vers les microservices et gestion de l'équilibrage de charge.
- **Types de Gateway :**
 - **Statique**

- **Dynamique** : utilise un **Discovery Service** pour localiser les services.
- **Configuration centralisée** : tous les microservices partagent un **fichier de configuration unique**, plutôt que chacun son propre fichier.

5/Gestion de la configuration centralisée

- **Configuration à froid** : lecture statique du fichier de configuration, performance faible.
- **Configuration à chaud** : lecture dynamique et mise à jour en temps réel, meilleure performance.
- **Services utilisés** : **Spring Cloud Config**, **Consul Config** pour centraliser et gérer les configurations des microservices.

6/Développement microservices avec Spring Cloud

- Exemple : équipe de 20 personnes gérant la scolarité d'une université.
- Chaque microservice est indépendant et responsable d'un module spécifique :
 - Étudiants : Entity, Repository, Service, REST Controller, DTO, Mapper
 - Classes, Notes, Modules, Salles : chaque module géré par un ingénieur différent.
- Granularité fine (principe PME) permet un **développement parallèle**.
- Avantage : même si un microservice prend 24h à être créé correctement, l'architecture microservices permet de **développer le backend global plus rapidement et de manière scalable**.

7/Démarrage et enregistrement des microservices

- Après création, le microservice est **démarré localement**.
- Il se connecte au **service de configuration** pour récupérer sa configuration.
- Ensuite, il s'enregistre auprès du **Discovery Service** avec son nom et son adresse.
- Pour chaque **périmètre fonctionnel**, on choisit la technologie la plus adaptée :
 - **AI / Machine Learning** : Python
 - **Temps réel** : Node.js
 - **Partie transactionnelle** : Java

8/Kubernetes et Docker

- **Kubernetes (K8s)** : système open source pour **automatiser le déploiement, la scalabilité et la gestion des conteneurs** sur des clusters de serveurs. Initialement développé par Google, il est maintenant géré par la **Cloud Native Computing Foundation**.
- **Docker** : plateforme lancée en 2013 permettant d'exécuter des applications dans des **conteneurs logiciels**, souvent utilisée avec Kubernetes.

9/Challenges, protocoles et communication dans les microservices

- **Challenges microservices** :
 - Communication entre microservices (via API, middleware, event bus).

- Gestion des bases de données : chaque microservice peut avoir sa propre DB.
- Techniques supplémentaires : brokers Kafka, frameworks Spring Cloud, conteneurs (Docker, Kubernetes), Big Data, cloud computing.
- **Protocoles réseau :**
 - **Orienté connexion** : établit, maintient et ferme un canal (ex. TCP, FTP).
 - **Non orienté connexion** : pas de canal ni de gestion de session (ex. Ethernet, IP, UDP, TFTP).
 - **Mode connecté** : session avec début, fin, validation d'erreurs, identifiant et mot de passe (couche 5 OSI, « session »).
 - **Mode non-connecté** : chaque paquet est autonome avec adresse de destination.
- **Communication :**
 - **Synchrone** : interaction en temps réel entre parties présentes.
 - **Asynchrone** : interaction décalée dans le temps, avec délai entre messages.
- **Kubernetes (K8s)** : plateforme open source pour automatiser **déploiement, scalabilité et gestion des conteneurs**, souvent utilisé avec Docker. Initialement conçu par Google, maintenant géré par la Cloud Native Computing Foundation.

MAGUEMOUN SAMY

Chapitre 4 : RESTful Web Services

Lien : <http://docmadhattan.fieldofscience.com/2011/11/birth-of-web.html>

1/REST (Representational State Transfer)

est un style d'architecture dérivé du Web.

Il repose sur un ensemble de **contraintes spécifiques**, ajoutées progressivement, pour définir un modèle d'architecture cohérent et efficace.

Approche de conception REST :

- On part d'un **modèle vide** (sans contraintes).
- Puis on applique **des contraintes du Web** de manière incrémentale pour façonner l'architecture REST.

Deux méthodes de conception d'architecture :

1. **Approche bottom-up** : partir d'une feuille blanche et assembler progressivement des composants connus jusqu'à satisfaire les besoins du système.
2. **Approche top-down (REST)** : partir des besoins globaux du système, puis ajouter progressivement des contraintes pour structurer l'architecture de façon naturelle et cohérente.

Idée clé de REST :

- Les contraintes ne sont pas arbitraires : elles guident l'architecture afin que le système s'organise efficacement, en harmonie avec les principes du Web.

2/Modèle vide (Null State)

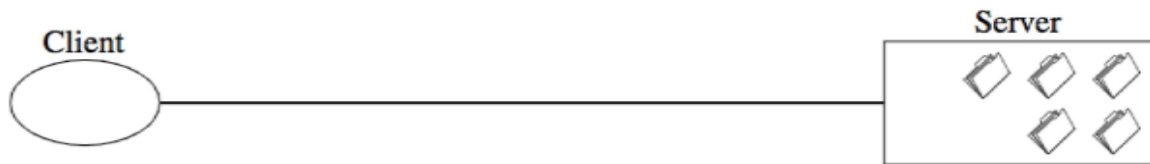
- Le **modèle vide (Null State)** représente un système **sans aucune contrainte architecturale**.
- Dans ce modèle :
 - Il n'existe **aucune règle** imposée sur la structure ou le fonctionnement du système.
 - Il n'y a **pas de frontières claires** entre les composants sur le plan architectural.
- **Pour REST** :
 - Le **World Wide Web (WWW)** est considéré comme le **Null State** de REST.
 - REST est obtenu en **ajoutant progressivement des contraintes** à ce modèle de base.
- **Idée clé** :
 - Le Null State sert de point de départ ; ce sont les contraintes REST qui donnent ensuite une architecture claire, structurée et efficace.

3/Client-Serveur

- La première contrainte ajoutée au modèle REST est le **modèle client-serveur**.
- Il sépare :
 - **Le client** : gestion de l'interface utilisateur.
 - **Le serveur** : gestion des données et de la logique.

Avantages :

- **Portabilité accrue des interfaces** (web, mobile, desktop).
- **Scalabilité facilitée** côté serveur.
- **Évolution indépendante** des composants client et serveur.



4/Sans État (Stateless)

- Chaque requête client doit contenir **toutes les informations nécessaires**.
- Le serveur **ne conserve aucun état** entre deux requêtes.
- Les requêtes sont **indépendantes les unes des autres**.

Avantages :

- Meilleure **visibilité, fiabilité et scalabilité**.
- Mise en œuvre plus simple côté serveur.

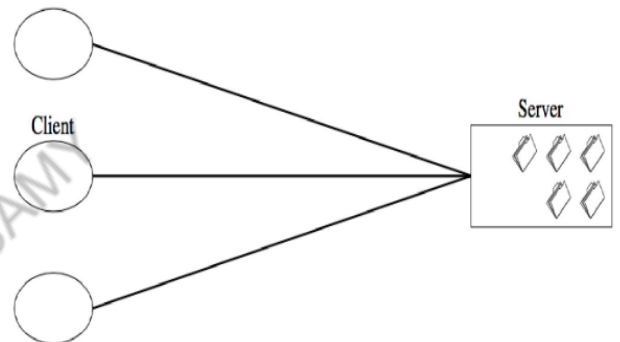


Figure 5-3. Client-Stateless-Server

Inconvénients :

- **Surcoût réseau** (répétition des données).
- **Moins de cohérence** et de contrôle côté serveur.

5/Cache (REST)

- Utilisé avec le modèle **client/serveur sans état** pour améliorer les performances.
- Les réponses doivent indiquer si elles sont **cachables ou non**.
- Réduit les accès serveur et **accélère les réponses**
- Le cache améliore les **performances, la scalabilité et l'efficacité** du système.
- Les contraintes ajoutées au Web servent à **guider son évolution** vers une architecture moderne.

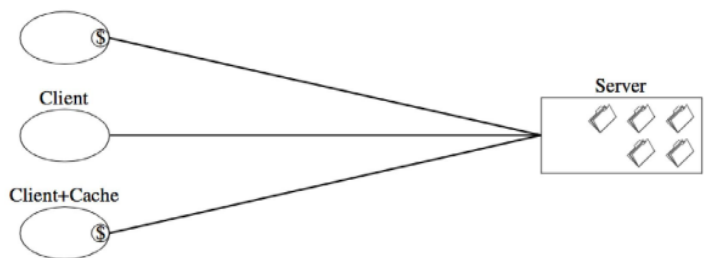
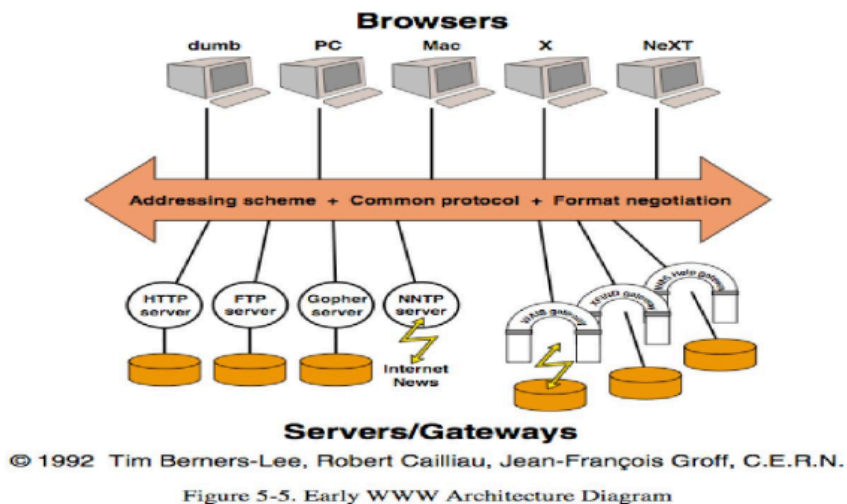


Figure 5-4. Client-Cache-Stateless-Server



Explication de l'image

Ce diagramme (1992) montre l'architecture initiale du World Wide Web et son objectif principal : **unifier l'accès à des ressources Internet hétérogènes**.

- **Navigateurs (clients)** : Fonctionnent sur différentes plateformes (PC, Mac, Unix, NeXT, terminaux simples), sans dépendre du système.
- **Cœur du Web** :
 - **Adressage universel** (URL/URI)
 - **Protocole commun** (HTTP)
 - **Négociation de formats** (choix du meilleur format via HTTP)
- **Serveurs et passerelles** :
 - Accès aux services existants (HTTP, FTP, Gopher, NNTP).
 - Les **gateways** traduisent HTTP vers d'autres protocoles pour les rendre accessibles via un navigateur.

Importance :

Le Web n'a pas remplacé immédiatement les systèmes existants, il les a **intégrés**. Cette unification (un navigateur, un protocole, des liens) a permis l'adoption rapide du Web et posé les bases du Web moderne.

6/Interface uniforme

L'**interface uniforme** impose des règles communes d'accès aux services.

Elle **découple les implémentations des services**, ce qui :

- facilite l'évolution indépendante des composants,
- améliore la maintenabilité et l'interopérabilité.

Inconvénient : cette standardisation peut **réduire l'efficacité** en limitant les optimisations spécifiques.

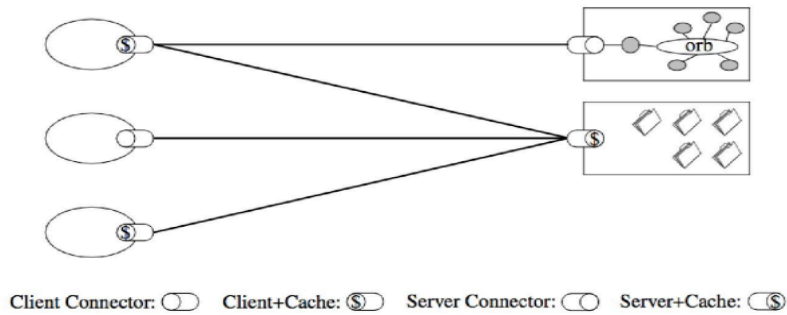


Figure 5-6. Uniform-Client-Cache-Stateless-Server

Une interface REST repose sur **quatre contraintes principales** :

1. **Identification des ressources**
Chaque ressource est identifiée de manière unique par une URI.
2. **Manipulation via des représentations**
Les ressources sont manipulées à travers leurs représentations (JSON, XML, etc.).
3. **Messages auto-descriptifs**
Chaque message contient toutes les informations nécessaires à sa compréhension.
4. **Hypermédia (HATEOAS)**
Les liens hypermédia guident les actions possibles et l'évolution de l'état de l'application.

Ces contraintes garantissent une interface uniforme, évolutive et indépendante des implémentations.

7/Système en couches

Un **système en couches** limite la connaissance du système à chaque couche, ce qui réduit la complexité globale et favorise l'indépendance entre les couches.

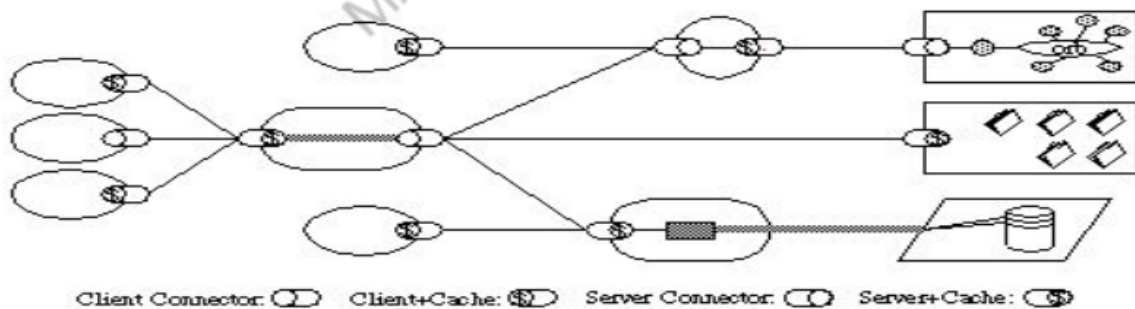


Figure 5-7. Uniform-Layered-Client-Cache-Stateless-Server

Les systèmes en couches ajoutent de la latence et un surcoût, mais l'utilisation de **caches intermédiaires** peut améliorer les performances. Ces couches facilitent aussi le renforcement des **politiques de sécurité** des données.

8/Code à la demande

Le **Code à la demande** permet à un client REST de télécharger et exécuter du code pour étendre ses fonctionnalités après déploiement. Cela améliore l'**extensibilité** mais réduit la **visibilité**. C'est donc une **contrainte optionnelle** dans REST.

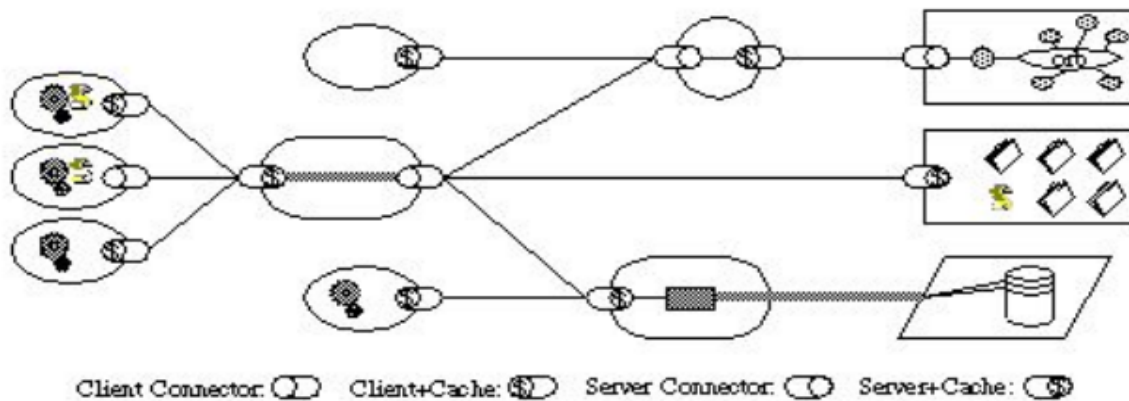


Figure 3-8. REST

REST est indépendant des technologies ou protocoles utilisés. Il définit les contraintes sur les **composants, connecteurs et données**, constituant la base de l'architecture du Web et le comportement des applications réseau.

9/Éléments de données

- **Ressource** : la cible conceptuelle d'une référence hypertexte
- **Identifiant de ressource** : URL, URN
- **Représentation** : document HTML, image JPEG
- **Méta-données de la représentation** : type de média, date de dernière modification
- **Méta-données de la ressource** : liens vers la source, liens alternatifs, variations
- **Données de contrôle** : if-modified-since, cache-control

10/Représentations REST

- Les actions sur une ressource se font via sa **représentation**, qui capture son état actuel ou prévu.
- Une représentation = **séquence d'octets + métadonnées descriptives**.
- Termes génériques pour désigner une représentation : document, fichier, entité HTTP, instance, variante.

Une représentation contient :

- **Données**
- **Métadonnées décrivant les données**
- Parfois, **métadonnées sur les métadonnées** (ex. pour vérifier l'intégrité)

Les métadonnées sont des **paires nom/valeur**.

Les messages peuvent inclure :

- Métadonnées de **représentation**
- Métadonnées de **ressource**

Données de contrôle : définissent la nature du message (action demandée, signification de la réponse, comportement du cache, etc.).

11/Connecteurs REST et exemples

REST utilise différents connecteurs pour gérer l'accès aux ressources et le transfert des représentations :

- **Client** : bibliothèque pour accéder aux ressources depuis le client
 - Exemples : `libwww`, `libwww-perl`
- **Serveur** : interface pour exposer les ressources
 - Exemples : `libwww`, `Apache API`, `NSAPI`
- **Cache** : pour stocker temporairement les réponses et améliorer les performances
 - Exemples : cache du navigateur, réseau de cache comme Akamai
- **Résolveur** : pour résoudre les noms de ressources (DNS)
 - Exemple : `bind`
- **Tunnel** : pour sécuriser ou relayer les connexions
 - Exemples : `SOCKS`, `SSL` après `HTTP CONNECT`

12/Ressources REST

Une ressource REST est tout élément accessible et transférable sur le Web.
C'est une correspondance logique et temporelle à un concept du domaine traité.

Exemples :

- Un reportage d'actualité
- La température à New York à 16h EST
- Une déclaration de revenus dans les bases de l'IRS
- Un résultat de recherche dans un index Web (ex. Google)

13/Représentation REST

La **représentation** d'une ressource est ce qui est échangé entre clients et serveurs.
C'est l'état temporaire des données réelles au moment de la requête.

Elle se compose :

- D'un **flux binaire** (les données)
- De **métadonnées** décrivant comment le client ou le serveur doit traiter ce flux (ex. type de média, validation, chiffrement, ou code à exécuter).

Une même ressource peut avoir plusieurs **représentations** selon le client :

- Image, fichier texte, flux XML ou JSON.
- Toutes accessibles via **la même URI**.

Exemples :

- Pour un navigateur humain → HTML.
- Pour un service automatisé → XML ou JSON, optimisé pour le traitement plutôt que la lisibilité.

URI : lien unique vers une ressource, permettant l'échange de représentations entre client et serveur.

Mapping CRUD → HTTP :

- **Create** → POST
- **Retrieve** → GET
- **Update** → PUT
- **Delete** → DELETE

MAGUEMOUN SAMY

Chapitre 5 : Inversion de Contrôle et Injection de Dépendances

Principe : concevoir des applications fermées à la modification mais ouvertes à l'extension.

1/Structure générale d'une application :

- **Frontend** :
 - Web : HTML, CSS, JavaScript
 - Desktop : Java AWT, Swing, JavaFX (ex : Word, Excel, PowerPoint)
 - Mobile : frameworks mobiles
- **Backend** : PHP, Java EE, Node.js, .NET, Python

Frameworks courants Java EE : Spring, Spring Security, Hibernate

- **Frameworks CSS/JS** :
 - CSS : Bootstrap
 - JS : Angular, Vue.js, React
- **Machine Learning** : Python
- **Comparaisons** :
 - Java vs JS : choix selon besoins back-end ou front-end
 - Java vs Python : choix selon performance, machine learning ou rapidité de développement
- **Concepts clés** :
 - Besoin fonctionnel : besoin métier
 - Besoin technique : justifie le choix de technologies (PHP, Node.js, Java...)
- **Déploiement en production** : mettre l'application sur serveur et l'exploiter avec de vrais utilisateurs

Problèmes possibles :

- Application mal conçue
- Application mal développée
- Application mal déployée (sur une seule ou plusieurs machines)

Exemples de sites/applications performants :

- Google : 89,3 milliards de recherches en 2021
- ChatGPT : 10 millions de requêtes par jour
- Netflix : lecture vidéo plus rapide en streaming que depuis un disque dur local

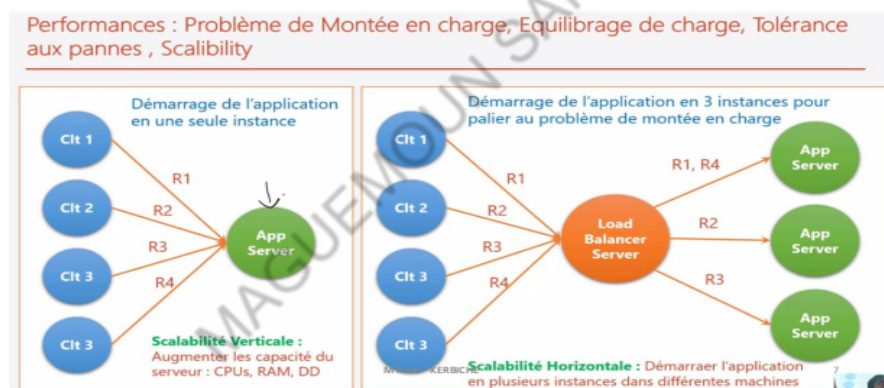
2/Netflix et aspects techniques

- **Technologie utilisée** :
 - Node.js pour client et serveur backend (plus simple à lire/écrire que Java)
 - Java reste utilisé pour le cœur du backend
- **Approche technique** :
 - Objectif principal : maîtriser un langage, un framework ou une technologie
 - Prendre en compte les besoins fonctionnels et la montée en charge

- **Solution pour montée en charge :**
 - Utiliser des architectures scalables et élast

3/Scalabilité et Kubernetes

- **Types de scalabilité :**
 - **Verticale** : augmenter CPU, RAM pour accélérer l'application.
 - **Horizontale** : déployer plusieurs instances sur plusieurs machines avec un **load balancer**.
- **Gestion automatique avec conteneurs :**
 - Les applications sont lancées dans des conteneurs (ex. Docker).
 - **Kubernetes** surveille les conteneurs : redémarre les pannes et ajuste le nombre de conteneurs selon la charge.
 - L'architecture peut être **élastique** : démarrage/arrêt virtuel de conteneurs selon besoin.
- **Haute disponibilité** : Kubernetes utilise des **clusters** (copies) pour éviter les pannes.
- **Résolution des problèmes de performance :**
 - Par l'**architecture technique** de l'application.
 - Par l'**architecture de déploiement** (scalabilité, load balancing, clusters).



4/Scalabilité verticale et technologies

- **Scalabilité verticale** : augmenter les ressources (CPU, RAM) pour gérer plus de requêtes.
- **Exemple comparatif :**
 - **PHP** : multi-threading → pour 10 000 requêtes, crée 10 000 threads → limité techniquement.
 - **Node.js** : single-thread avec E/S non bloquantes → peut gérer 10 000 requêtes avec un seul thread, limité seulement par CPU/mémoire.
- **Conclusion** : Node.js gère mieux la scalabilité verticale que PHP.

5/Application fermée à la modification et ouverte à l'extension

- **Principe** : Une bonne application est **fermée à la modification** (le code source n'est pas touché pour les évolutions ou corrections) et **ouverte à l'extension** (ex. plugins, modules additionnels).
- **Avantages** :

1. Facilite la maintenance et l'évolution sans risque sur le code existant.
 2. Permet la TMA (Tierce Maintenance Applicative) par un prestataire tiers.
- **Risques si le code source est modifié :**
 1. **Effets de régression** : modification d'une partie du code peut casser d'autres fonctionnalités.
 2. **Difficulté de compréhension** pour un nouveau développeur.
 3. **Maintenance compliquée**, surtout sans tests unitaires.
 - **Exemple ERP** : Zoho, SAP, Sage, Oracle, NetSuite, Cegid, Microsoft Dynamics, Divalto, WaveSoft, Odoo, Archipelia.

MAGUEMOUN SAMY

Chapitre 6 : SOA In Practice

1/Motivation

- Les systèmes sont grands et hétérogènes, ce qui nécessite **flexibilité** et **communication**.
- SOA répond à cette complexité en permettant une meilleure adaptation.
- Principe inspiré du **darwinisme** :
ce ne sont pas les systèmes les plus forts ou les plus intelligents qui survivent, mais ceux **qui s'adaptent le mieux au changement**.

2/ Caractéristiques des grands systèmes distribués

- Les grands systèmes doivent gérer l'**existant (legacy)** :
on ne peut pas tout reconstruire à zéro. SOA s'introduit progressivement dans des systèmes déjà en production, en respectant la **rétro-compatibilité** et les anciennes plateformes.
- SOA n'est pas un simple projet ponctuel :
c'est une **transformation structurelle** d'un système existant.
- Les grands systèmes sont **hétérogènes par nature** :
langages, plateformes et systèmes d'exploitation différents.
- Ils sont aussi **complexes** :
comprendre l'impact des changements et choisir où intervenir est difficile.
- Enfin, ils sont **imparfaits** :
viser la perfection coûte trop cher.
Au-delà d'un certain seuil de complexité, l'effort augmente beaucoup plus vite que les bénéfices (courbe en « hockey stick »), ce qui peut mener à l'échec du système.



FIGURE 1-1. The hockey stick function

Explication de l'image – Courbe « hockey stick » (complexité vs effort)

- À **faible ou moyenne complexité**, l'effort reste **stable et maîtrisable** : on peut ajouter des fonctionnalités sans surcoût majeur.
- À un **seuil critique** (point d'inflexion), la situation change.

- Au-delà de ce seuil, l'effort **augmente très rapidement** (maintenance, bugs, interactions), devenant **coûteux et difficile à contrôler**.
- Cette courbe explique pourquoi il faut **limiter la complexité**, privilégier des designs simples et le **refactoring**.
- Elle est utilisée en ingénierie logicielle (ex. **COCOMO**) pour montrer que **plus la complexité augmente, plus l'effort explose**.

3/ L'Histoire du Bus Magique

- Dans un système avec **n composants**, une approche classique nécessite jusqu'à **$n \times (n-1)/2$ connexions** pour que chaque système communique avec tous les autres.
- Cela montre que **la complexité des connexions croît rapidement** avec le nombre de systèmes.

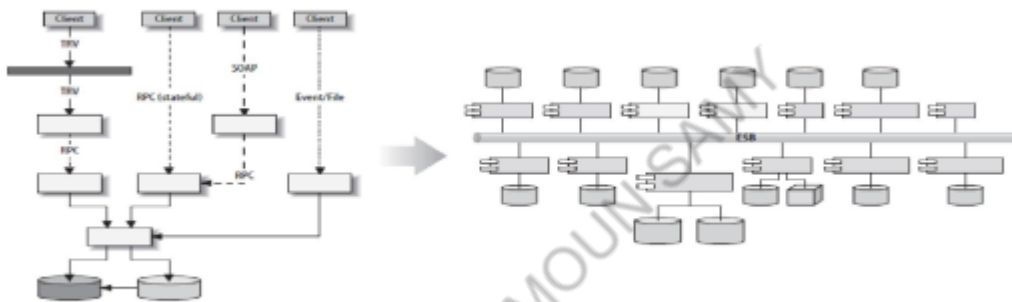


FIGURE 1-2. The original drawing of the Magic Bus

Explication de l'image -The Magic Bus (FIGURE 1-2)

- **Vision idéalisée (à gauche)** : un seul bus central ("Magic Bus") connecte tous les clients via différents protocoles (TM, RPC, SOAP, Event/Tib). Solution simple et "magique".
- **Réalité (à droite)** : en pratique, l'ESB devient un enchevêtrement complexe d'adaptateurs, routes et transformations — difficile à maintenir et à faire évoluer.
- **Message clé** : Les solutions centralisées tout-en-un paraissent élégantes sur le papier mais créent une complexité cachée ; d'où l'intérêt des architectures décentralisées et légères comme les microservices et la messagerie asynchrone.

4/SOA

- **Services** : fonctionnalités métier autonomes, réutilisables dans plusieurs processus, implémentables sur n'importe quelle technologie ou plateforme.
- **Infrastructure spécifique** : Enterprise Service Bus (ESB) pour combiner les services de manière flexible.

- **Politiques et processus** : gèrent l'hétérogénéité, la maintenance et la multiplicité des propriétaires dans les systèmes distribués.
- **Principe clé** : SOA favorise flexibilité, décentralisation et tolérance aux pannes dans les systèmes distribués complexes.

5/SOA en cinq diapositives

1. **SOA** : Paradigme pour gérer et maintenir des processus métier dans de grands systèmes distribués, basé sur les services, l'interopérabilité via un ESB et le couplage lâche.
2. **Politiques & Processes** : La distribution modifie l'organisation ; la mise en place de nouvelles fonctionnalités implique la collaboration de multiples systèmes et équipes.
3. **Web Services** : Une des façons techniques de réaliser SOA, mais SOA dépasse ses seuls aspects techniques.
4. **SOA in Practice** : Les concepts théoriques peuvent diverger de la pratique à cause de contraintes réelles (performance, sécurité, etc.).
5. **Governance & Management** : L'aspect clé de SOA est de définir la gouvernance appropriée : bonnes équipes, bonnes personnes, et gestion adaptée.

6/SOA –Terminology

- **Provider** : système qui implémente un service pour le rendre utilisable par d'autres systèmes.
- **Consumer** : système qui utilise ou appelle un service fourni par un provider.

7/SOA – Services

- **Services représentant une fonctionnalité métier**: Chaque service correspond à une fonctionnalité métier élémentaire (Elementary Business Process – EBP).
- **Interfaces et contrats** : Les services exposent des interfaces définies par des contrats, décrivant comment les consommateurs peuvent interagir avec eux.
- **Attributs supplémentaires des services** :
 - **Autonome (Self-Contained)** : Le service est autonome, ne dépend pas d'autres services pour fonctionner.
 - **Granularité élevée (Coarse-Grained)** : Fournit des fonctionnalités larges plutôt que des opérations trop fines.

- **Visible / Découvrable (Visible/Discoverable)**: Le service peut être découvert et utilisé par d'autres systèmes.
- **Sans état (Stateless)**: Idéalement, le service ne conserve pas d'état entre les appels pour faciliter la scalabilité.
- **Idempotent** : Les appels répétés produisent le même effet qu'un seul appel, pour éviter des effets secondaires indésirables.

MAGUEMOUN SAMY

Chapitre 7 : SOAP Web Services

Les **services web** sont une infrastructure permettant de développer des applications **interopérables sur le Web**.

Ils sont **étroitement liés à l'architecture SOA** et reposent largement sur l'utilisation de **XML** pour l'échange de données.

1/SOAP Web Service

Un service Web SOAP fonctionne en plusieurs étapes :

Définition du service (fournisseur) : le fournisseur décrit le service avec un **WSDL** (opérations, entrées/sorties).

Publication du service (fournisseur) : le fournisseur publie le WSDL dans un **annuaire UDDI**.

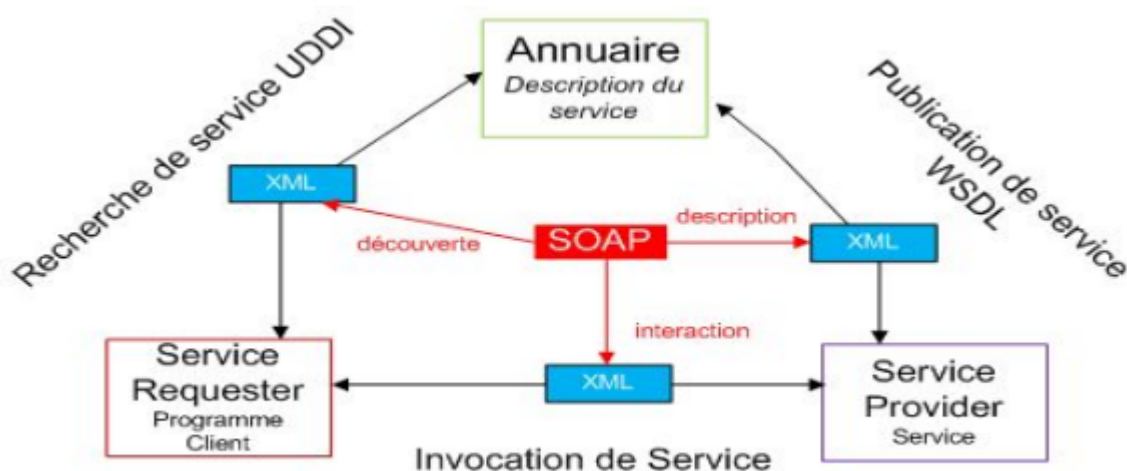
Recherche de service(client): le client cherche un service dans l'annuaire et récupère son adresse.

Enregistrement au service web (client)(optionnel) : le client s'enregistre auprès du fournisseur.

Appel du service (client) : le client appelle le service via des messages **SOAP** avec des paramètres.

Composition (client,fournisseur) : le résultat du service est utilisé pour appeler d'autres services (côté client ou fournisseur).

En résumé : **définir** → **publier** → **rechercher** → **appeler** → **composer**.



Explication de l'image

Le schéma illustre le **modèle SOA en triangle** basé sur le principe **Publish – Find – Bind**.

- **Service Provider** : implémente et expose le service.
- **Service Requester** : client qui consomme le service.
- **Annuaire (UDDI)** : registre où les services sont décrits et publiés.

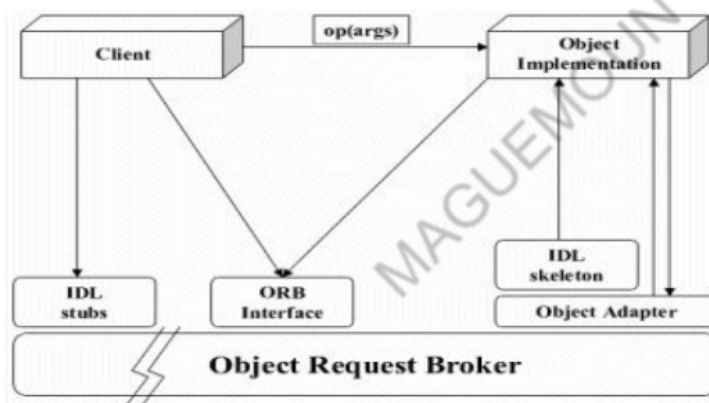
Fonctionnement :

1. Le **Provider publie** la description du service (**WSDL**) dans l'annuaire.
2. Le **Requester recherche** un service dans l'annuaire et récupère le WSDL.
3. Le **Requester invoque** ensuite le service directement auprès du Provider via **SOAP**.

Idée clé :

Ce modèle permet une **intégration lâche**, une **découverte dynamique** et l'interopérabilité entre systèmes hétérogènes, fondements des Web Services SOA classiques.

2/Comparaison avec les solutions middleware (CORBA)



- **Principe général** : CORBA permet la communication entre objets distribués via un intermédiaire central appelé **ORB (Object Request Broker)**.
- **Transparence** : le client appelle une méthode comme si l'objet était local ; l'ORB masque le réseau, la localisation et le langage.
- **Description d'interface** : les services sont décrits en **IDL (Interface Definition Language)**, indépendante des langages.
- **Côté client** :
 - utilisation d'un **stub** (proxy généré à partir de l'IDL) ;

- le stub sérialise les paramètres et envoie la requête à l'ORB.
- **Côté serveur :**
 - un **skeleton** reçoit la requête via l'ORB ;
 - il désérialise les paramètres et appelle l'implémentation réelle de l'objet.
- **Communication** : assurée par l'ORB (souvent via le protocole **IIOF**).
- **Retour de résultat** : chemin inverse (objet → skeleton → ORB → stub → client).
- **Avantages** : interopérabilité multi-langages, forte abstraction, appels distants transparents.
- **Limites** : infrastructure lourde et complexe par rapport aux approches modernes (REST, microservices).

Idée clé : CORBA est un ancêtre des mécanismes actuels de communication distribuée (RMI, gRPC, clients REST générés), basé sur des proxies automatiques et une interface neutre.

3/Technologies de base des Web Services

- **SOAP (Simple Object Access Protocol)** :
Protocole de communication permettant d'appeler des services web à distance (RPC) via l'échange de messages XML.
- **WSDL (Web Service Description Language)** :
Langage de description des services web. Il définit les opérations disponibles, les paramètres, les types de données, les résultats et les points d'accès du service.
- **UDDI (Universal Description, Discovery and Integration)** :
Mécanisme d'annuaire permettant de publier, décrire et découvrir des services web.

4/SOAP

- **Définition** :
SOAP (Simple Object Access Protocol) est une norme du W3C pour l'échange de messages entre applications.
- **Évolution** :
 - SOAP 1.0 (1999) : basé sur HTTP
 - SOAP 1.1 (2000) : plus générique, supporte d'autres protocoles
 - SOAP 1.2 (2007) : recommandation officielle du W3C

- **Aspects couverts par SOAP :**

- Format XML des messages échangés
- Transport des messages (HTTP, SMTP, etc.)
- Règles de traitement des messages
- Conversion des appels RPC en messages SOAP

- **Objectifs :**

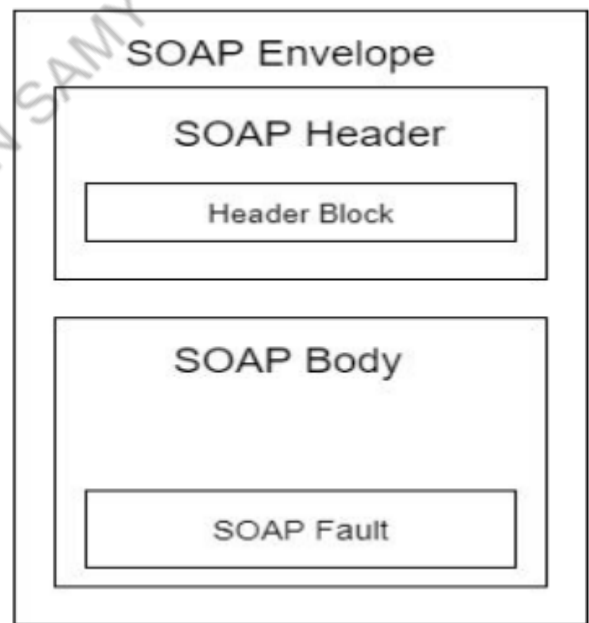
- Fournir une infrastructure minimale pour le RPC sur le Web
- Utiliser XML pour les échanges
- S'appuyer sur HTTP pour traverser les pare-feu
- Éviter les limites de CORBA sur le Web
- Faciliter l'interopérabilité entre systèmes et plateformes hétérogènes
- Évoluer vers un support générique d'échange de messages, au-delà du RPC

- **Structure d'un message SOAP :**

- **Enveloppe** : conteneur XML du message
- **En-tête (optionnel)** : informations techniques (sécurité, routage, etc.)
- **Corps (obligatoire)** : données applicatives échangées

Explication de l'image -Structure d'une enveloppe SOAP

- Un message SOAP est toujours contenu dans une **SOAP Enveloppe** (enveloppe XML).
- Elle se compose de deux parties principales :



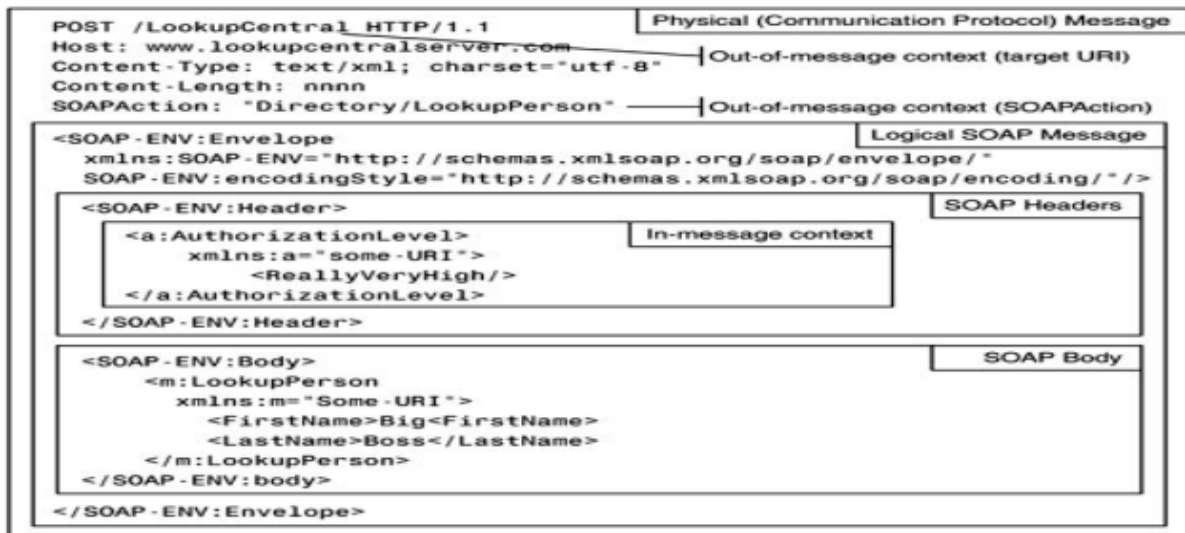
1. SOAP Header (optionnel)

- Contient des blocs d'en-tête.
- Sert aux métadonnées techniques : sécurité, authentification, transactions, routage, journalisation, etc.
- Extensible et indépendant des données métier.

2. SOAP Body (obligatoire)

- Contient les données applicatives (requête ou réponse).
- Transporte les paramètres et les résultats du service.
- Peut inclure un **SOAP Fault** pour décrire une erreur (code, message, détails).

Exemple :



11

En-tête SOAP

- Conteneur d'informations indépendantes de l'application (XML).
- Contient des blocs pour : sécurité, coordination, identifiants de transaction, etc.
- Chaque nœud intermédiaire peut traiter certains blocs.
- **Attributs clés :**
 - **mustUnderstand :**
 - **true** → traitement obligatoire, sinon erreur.
 - **false** → traitement optionnel (par défaut).
 - **relay** (SOAP 1.2) : si non traité, le bloc est transmis au nœud suivant.

Cheminement des messages SOAP

- Les messages suivent un chemin source → destination dans le réseau, prenant en compte l'architecture (middleware).
- Chaque bloc d'en-tête a un **role** (par défaut : **ultimateReceiver**).
- Chaque nœud :
 - Traite les blocs d'en-tête qui le concernent.
 - Respecte **mustUnderstand="true"** pour les blocs obligatoires.
 - Supprime ou retransmet les blocs selon l'attribut **relay**.
- Le **corps du message** n'est traité que par le destinataire final.
- En cas d'erreur : arrêt du cheminement et retour d'un message SOAP Fault.
- Les **intermédiaires actifs** peuvent modifier les en-têtes de façon dépendante de l'application.

Corps SOAP

- Contient les **données spécifiques à l'application**.
- Composé de **blocs** :
 - Chaque bloc est traité uniquement par le **destinataire final**.

- Certains blocs ont des fonctions prédéfinies :
 - Traduction d'un appel RPC en SOAP.
 - Signalement d'erreurs via un **bloc Fault** :
 - **Code** : catégorie d'erreur (version, mustUnderstand, client, server).
 - **Texte** : explication lisible.
 - **Acteur** : nœud à l'origine de l'erreur.
 - **Détail** : informations spécifiques à l'application.

Appel RPC en SOAP

- **Principe** : Utilise deux messages SOAP : **appel** et **réponse**.
 - **Message d'appel** :
 - Élément : nom de la fonction à appeler.
 - Sous-éléments : paramètres de la fonction.
 - **Message de retour** :
 - Résultat de l'appel.
 - Ou élément **Fault** pour signaler une erreur.
- **Transport** : Généralement via HTTP (GET/POST), mais pas obligatoire.

Exemple : Message d'appel

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">

  <soapenv:Body>

    <add
soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"

      <op1 xsi:type="xsd:int">2</op1>

      <op2 xsi:type="xsd:int">5</op2>

    </add>

  </soapenv:Body>

</soapenv:Envelope>
```

Exemple : Message réponse

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<soapenv:Body> <addResponse
soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<addReturn xsi:type="xsd:int">7</addReturn> </addResponse>
</soapenv:Body> </soapenv:Envelope>
```

5/SOAP et HTTP

- **Binding SOAP** : décrit comment les messages SOAP sont transmis via un protocole de transport.
- **SOAP sur HTTP** : utilise **GET** ou **POST**.
 - **GET** : seul le message de réponse est SOAP, l'appel ne l'est pas.
 - **POST** (préféré) : l'appel et la réponse sont tous deux des messages SOAP.
- **Inclusion dans HTTP** :
 - **Appel POST** : enveloppe SOAP = contenu du message POST.
 - **Réponse** : enveloppe SOAP = contenu de la réponse HTTP.
- **Gestion des erreurs** : SOAP utilise les mêmes codes d'état et d'erreur que HTTP.

6/WSDL

- **Définition** : WSDL (Web Services Description Language) décrit un Service Web.
 - Écrit en **XML**
 - Extension : **.wsdl**
 - Media type : **application/wsdl+xml**
 - Développé par le **W3C**

Structure d'un document WSDL :

```
<definitions>
```

```
<types>
```

```
data type definitions.....
```

```

</types>

<message>
    definition of the data being communicated....
</message>

<portType>
    set of operations.....
</portType>

<binding>
    protocol and data format specification....
</binding>
</definitions>

```

Éléments principaux :

Élément	Description
<types>	Définit les types de données XML utilisés par le service
<message>	Définit les données pour chaque opération
<portType>	Décrit les opérations disponibles et les messages associés
<binding>	Définit le protocole et le format de données pour chaque portType

Le <portType>

- Définit un service web, ses opérations et les messages associés.
- Types d'opérations :

Type	Définition
One-way	Reçoit un message, pas de réponse
Request-response	Reçoit une requête et retourne une réponse
Solicit-response	Envoie une requête et attend une réponse
Notification	Envoie un message sans attendre de réponse

WSDL Binding pour SOAP

```

<binding type="points to the port for the binding" name="anyname">

  <soap:binding style="document"
transport="http://schemas.xmlsoap.org/soap/http" />

  <operation>

    <soap:operation soapAction="http://example.com/getNoteEt"/>

    <input><soap:body use="literal"/></input>

    <output><soap:body use="literal"/></output>

  </operation>

</binding>

```

- **<binding>** : attributs **name** (nom du binding) et **type** (référence au portType)
- **<soap:binding>** : attributs **style** (**rpc** ou **document**) et **transport** (HTTP, SMTP...)
- **<operation>** : définit chaque opération exposée par le **portType**

7/UDDI

- **Définition** : Universal Description, Discovery and Integration
 - Annuaire universel pour les services web (comme un Google des services web)
 - Chaque entrée est un document XML avec :
 - **businessEntity** : organisation offrant le service
 - **businessService** : liste des services web proposés par l'organisation
 - **bindingTemplate** : aspects techniques du service
 - **tModel** : informations génériques supplémentaires sur le service
- **Types d'information dans l'annuaire** :
 - **Pages blanches** : données sur le fournisseur (nom, adresse...)
 - **Pages jaunes** : classification du type de service selon des standards
 - **Pages vertes** : informations techniques sur l'utilisation du service

MAGUEMOUN SAMY